

Module 13: API Gateway Filters, Centralized Configuration, Distributed Tracing, and Centralized Logging

Questions

Q1. What are API Gateway filters, and why are they important in a microservices architecture?

Answer: API Gateway filters are components that process requests and responses as they pass through the gateway. They are important because they centralize cross-cutting concerns that every service would otherwise have to implement itself. Filters can modify requests before they reach downstream services, modify responses before they return to clients, reject requests, log traffic, add security headers, enforce rate limits, and handle circuit breaker fallbacks. In a microservices architecture, this centralization reduces duplication and makes it easier to enforce consistent policies such as authentication, logging, and traffic management across all services.

Q2. What is the difference between a `GatewayFilter` and a `GlobalFilter` in Spring Cloud Gateway?

Answer: A `GatewayFilter` is route-specific and is attached to individual routes through filter factories. It modifies requests or responses only for the routes where it is configured. A `GlobalFilter` applies to all routes and is used for cross-cutting concerns such as routing, load balancing, metrics, and logging. Both implement the same `filter(exchange, chain)` signature and participate in the same ordered filter chain. Spring Cloud Gateway merges route-specific `GatewayFilter` instances with global filters at runtime, ordered by `Ordered`. The highest precedence filter runs first in the pre phase and last in the post phase.

Q3. What are default filters in Spring Cloud Gateway?

Answer: Default filters are filters configured under `spring.cloud.gateway.default-filters` that apply to every route automatically. They are useful for adding common behavior such as a response header, a prefix path, or a correlation ID without repeating the filter in every route definition. For example, you can add a default response header that applies to all responses, or a default request header that downstream services can use for tracing. Default filters are merged with route-specific filters for each route, so they behave as if they were declared on every route.

Q4. How do you write a custom `GatewayFilterFactory` in Spring Cloud Gateway?

Answer: You extend `AbstractGatewayFilterFactory<T>` where `T` is a configuration class that holds filter properties. The class name must end with `GatewayFilterFactory`, for example `JwtAuthGatewayFilterFactory`, and it is referenced in YAML by the short name `JwtAuth`. Inside the factory, you implement the `apply` method to return a `GatewayFilter` that performs the desired logic using `ServerWebExchange`. The filter can inspect or modify the request, decide whether to proceed, and transform the response. Custom filter factories are useful when a filter needs configuration per route and should behave like a built-in gateway filter.

Q5. How do you write a custom `GlobalFilter` for authentication in Spring Cloud Gateway?

Answer: You create a class that implements both `GlobalFilter` and `Ordered`, and register it as a bean. In the `filter` method, you inspect the incoming request, for example by reading the `Authorization` header, validate the token, and either continue the chain or return an unauthorized response. Because Spring Cloud Gateway is reactive, all operations must return `Mono<Void>` and avoid blocking the event loop. If authentication succeeds, you can add user information to the request headers so downstream services receive a trusted identity. Ordering matters: the authentication filter should run before routing filters so unauthenticated requests are rejected early.

Q6. What is the `TokenRelay` filter, and how does it help with authentication propagation?

Answer: `TokenRelay` is a built-in Spring Cloud Gateway filter that forwards an OAuth2 access token from the authenticated gateway session to downstream services. When a user logs in through the gateway using OAuth2, the gateway holds the access token in the security context. The `TokenRelay` filter copies that token into the `Authorization` header of the proxied request so downstream

services can validate the same token. This avoids requiring users to authenticate separately with each service and centralizes token handling at the gateway. It requires the OAuth2 client dependency and proper `spring.security.oauth2.client` configuration.

Q7. Why should JWT validation happen at the API Gateway instead of in every microservice?

Answer: Validating JWT at the gateway centralizes security logic, reduces duplication, and prevents every downstream service from needing token parsing libraries, public key configuration, and token expiration handling. The gateway rejects invalid or expired tokens before traffic reaches the backend, which reduces load on services and gives clients consistent error responses. Downstream services can then trust headers added by the gateway, such as `X-User-Id` or `X-Roles`, though they should still perform their own authorization checks based on those claims. Centralization also makes key rotation simpler because only the gateway needs to know about the new public key.

Q8. What are common pitfalls when implementing authentication in Spring Cloud Gateway?

Answer: Common pitfalls include blocking the reactive event loop during token validation, which kills gateway throughput; assuming Spring Security context propagates automatically to downstream services without forwarding headers; caching JWK keys forever and failing when the identity provider rotates them; mixing servlet-based security code with the reactive gateway; and putting complex business authorization logic in the gateway instead of the services. Another mistake is not handling expired tokens cleanly, which can cause confusing errors for clients. The gateway should validate authentication and forward identity, while services decide what the authenticated user is allowed to do.

Q9. What is Spring Cloud Config Server, and why is it used?

Answer: Spring Cloud Config Server provides centralized externalized configuration for distributed systems. It serves configuration properties from a backend store, usually a Git repository, to client applications at runtime. Each microservice fetches its configuration by application name, profile, and label. Centralized configuration makes it easier to manage environment-specific settings, track changes through version control, and avoid embedding secrets or environment URLs in application packages. It is especially useful in microservices where dozens of services need consistent and auditable configuration.

Q10. How do you set up a Spring Cloud Config Server backed by GitHub?

Answer: You create a Spring Boot application, add `@EnableConfigServer` and the `spring-cloud-config-server` dependency, and configure `spring.cloud.config.server.git.uri` to point to the Git repository. Optionally, set `clone-on-start: true` so the server clones the repository at startup. The Git repository contains property files named after client applications, such as `order-service.properties` or `order-service-dev.yml`. The config server exposes endpoints at `/ {application} / {profile} / {label}`, for example `http://localhost:8888/order-service/dev/main`. Clients connect to the server using `spring.config.import=optional:configserver:http://localhost:8888/` and set `spring.application.name` to match the file name.

Q11. How does a Spring Cloud Config client load configuration at startup?

Answer: A config client connects to the Config Server during bootstrap, before the Spring context finishes initializing. It uses `spring.application.name` to determine which files to fetch, and the active profile to select environment-specific overrides. The configuration is merged with local `application.properties` or `application.yml`, with remote properties taking precedence. If the Config Server is unavailable, the client can fail, retry, or fall back depending on the `optional` setting in the import and failure settings. Once loaded, the properties are available to `@Value`, `@ConfigurationProperties`, and the `Environment` just like local properties.

Q12. What is `@RefreshScope`, and how does dynamic configuration refresh work?

Answer: `@RefreshScope` is a Spring Cloud annotation that marks a bean to be recreated when a refresh event occurs. When configuration changes in the Config Server repository, calling `POST /actuator/refresh` on a client triggers a refresh event. Spring recreates beans annotated with `@RefreshScope` so they pick up the new property values without restarting the JVM. Only beans that use the changed properties need the annotation; beans that do not need refresh should not use it to avoid unnecessary recreation. Dynamic refresh is useful for updating feature flags, timeouts, rate limits, and other non-disruptive settings in production.

Q13. How do you refresh configuration across all clients without restarting them?

Answer: To refresh all clients, you use Spring Cloud Bus combined with a message broker such as RabbitMQ or Kafka. When a property changes, you call `POST /actuator/busrefresh` on one client or the Config Server, and the event is broadcast to all connected clients through the bus. Each client then refreshes its own `@RefreshScope` beans. Alternatively, you can configure a GitHub webhook to notify the Config Server's `/monitor` endpoint whenever the repository changes, and the server publishes a refresh event to the bus. This push-based approach scales better than calling `/actuator/refresh` on every instance manually.

Q14. How does the Config Server handle updates in the Git repository?

Answer: By default, the Config Server clones or pulls the Git repository on each configuration request to ensure clients receive the latest values. The frequency can be controlled with the `refreshRate` property so the server does not pull on every request. When using Spring Cloud Bus, the `/monitor` endpoint can be triggered by a GitHub webhook on push events, forcing an immediate refresh across clients. The server maintains a local copy of the repository, so it can continue serving configuration even if GitHub is temporarily unreachable, though the data may be slightly stale.

Q15. What is distributed tracing, and why is it important in microservices?

Answer: Distributed tracing is the practice of tracking a request as it flows through multiple services. Each request is assigned a trace ID, and each unit of work within a service is assigned a span ID. These identifiers are propagated across service boundaries using HTTP headers and recorded in logs and trace collectors. Distributed tracing is important in microservices because a single user request may touch many services, and without tracing it is nearly impossible to pinpoint where latency or errors occur. Tracing helps debug slow requests, identify failing services, and understand dependencies in a distributed system.

Q16. What are trace ID and span ID, and how do they relate?

Answer: A trace ID is a unique identifier assigned to the entire request journey across all services. A span ID is a unique identifier for a single unit of work within one service, such as handling an HTTP request or calling a database. One trace contains many spans, and spans can be nested as parent and child to show the call hierarchy. For example, an order trace might have spans for the gateway, order service, payment client call, and database query. The trace ID ties them all together, while span IDs show the individual operations and their durations.

Q17. What is the difference between Spring Cloud Sleuth and Micrometer Tracing?

Answer: Spring Cloud Sleuth was the original distributed tracing solution in the Spring Cloud ecosystem. It automatically added trace and span IDs to logs and propagated them across services. In Spring Boot 3, Sleuth is deprecated and replaced by Micrometer Tracing, which is the vendor-neutral observation and tracing facade for Spring. Micrometer Tracing integrates with Brave for Zipkin or OpenTelemetry for other backends. The configuration namespace changed from `spring.sleuth.*` to `management.tracing.*`. Micrometer Tracing also provides better integration with metrics and the Observation API.

Q18. How do you configure distributed tracing with Micrometer and Zipkin in Spring Boot 3?

Answer: You add the `micrometer-tracing-bridge-brave` and `zipkin-reporter-brave` dependencies, along with `spring-boot-starter-actuator`. In `application.yml`, you set `management.tracing.sampling.probability` to control what fraction of requests are traced, and `management.zipkin.tracing.endpoint` to point to the Zipkin collector, such as `http://localhost:9411/api/v2/spans`. You also set `spring.application.name` so each service appears correctly in the Zipkin UI. The tracing library automatically propagates trace context through HTTP clients such as `RestTemplate`, `WebClient`, and OpenFeign when they are built through Spring's builders.

Q19. What are B3 and W3C trace context propagation formats?

Answer: B3 is the propagation format originally defined by Zipkin, using headers such as `X-B3-TraceId`, `X-B3-SpanId`, and `X-B3-ParentSpanId`. W3C trace context is a newer standard that uses a single `traceparent` header and an optional `tracestate` header. Both formats carry the trace and span identifiers across service boundaries. The choice depends on the ecosystem: older Zipkin-based systems often use B3, while OpenTelemetry and newer standards use W3C. In a mixed environment, services may need to support both formats to ensure propagation across all components.

Q20. How do you correlate logs with distributed tracing?

Answer: You configure the logging pattern to include the trace ID and span ID from the MDC, which Micrometer Tracing or Sleuth populates automatically. For example, the log pattern can include

`traceId=%X{traceId},spanId=%X{spanId}` . When logs are sent to a centralized system such as Elasticsearch or Splunk, you can filter by trace ID to see all log entries from every service that participated in the same request. This bridges the three pillars of observability: logs, metrics, and traces. The key is to propagate the trace context into every service and make sure the logging framework has access to it.

Q21. What is the ELK stack, and how does it enable centralized logging?

Answer: ELK stands for Elasticsearch, Logstash, and Kibana. Elasticsearch stores and indexes log data. Logstash collects, parses, enriches, and forwards logs from multiple sources into Elasticsearch. Kibana provides a web interface to search, visualize, and dashboard the logs. In a microservices environment, each service sends its logs to Logstash, either directly over TCP, through a file shipped by Filebeat, or through a logging appender. Once in Elasticsearch, all logs are searchable from a single place. This makes it possible to debug issues across services without logging into individual machines.

Q22. What is the role of Filebeat in the ELK stack?

Answer: Filebeat is a lightweight log shipper from Elastic that reads log files and sends them to Logstash or directly to Elasticsearch. In a Spring Boot microservices deployment, each container or host runs Filebeat, which monitors the application's log files and forwards new entries as they are written. Filebeat is more efficient than running a full Logstash agent on every host, and it handles backpressure and delivery guarantees. It is often combined with Logstash for parsing and enrichment, especially when logs need complex transformation before being indexed.

Q23. What is structured logging, and why is it useful with ELK?

Answer: Structured logging means writing log entries as JSON or another parseable format with consistent fields such as timestamp, level, logger, message, trace ID, and service name. It is useful with ELK because Logstash or Filebeat can parse the fields automatically, and Elasticsearch can index them for fast searching and aggregation. Without structured logs, log parsing is fragile and full-text search is slower. Spring Boot can output JSON logs using Logback or Log4j2 configurations. Structured logging also makes it easier to build dashboards in Kibana and to set alerts based on field values.

Q24. What are the alternatives to the ELK stack for centralized logging?

Answer: Alternatives include the EFK stack, which replaces Logstash with Fluentd; Grafana Loki, which is lightweight and integrates well with Grafana; Splunk, which is enterprise-grade and expensive; Datadog and New Relic, which combine logs, metrics, and traces in a hosted service; and cloud-native

solutions such as AWS CloudWatch Logs and Google Cloud Logging. The choice depends on cost, scale, existing tooling, and whether the team wants a self-hosted or managed solution. For many Kubernetes deployments, Loki is popular because it indexes only labels and stores compressed raw logs, making it cheaper than Elasticsearch at large scale.

Scenario based questions

S1. A Spring Cloud Gateway route needs to authenticate JWT tokens and forward user identity downstream. How do you implement it?

Answer: I implement a custom `GlobalFilter` that runs early in the filter chain and validates the JWT from the `Authorization` header. The validation should be non-blocking, using a reactive JWT library such as Spring Security OAuth2 Resource Server or Nimbus JOSE. If the token is invalid or missing, the filter returns 401 and stops the chain. If valid, it extracts claims such as `sub` and `roles`, and adds them as headers like `X-User-Id` and `X-User-Roles` to the downstream request by mutating `ServerWebExchange`. Downstream services can then trust these headers. I order the filter before routing so unauthenticated requests never reach the backend. I also cache the JSON Web Key set with a sensible TTL to avoid repeated fetches from the identity provider, and I handle key rotation by refreshing the cache on validation failures. Real-time example: imagine an e-commerce mobile app sends a request to `https://api.shop.com/orders`. The gateway validates the JWT, extracts the user ID `u-98231` and role `CUSTOMER`, then forwards the request to the order service with headers `X-User-Id: u-98231` and `X-User-Roles: CUSTOMER`. The order service does not need to talk to the identity provider; it simply trusts the gateway and enforces that the user can only see their own orders.

S2. A custom GatewayFilter works locally but is not applied to a route in production. What do you check?

Answer: I first check the class name. For Spring Cloud Gateway to reference a filter by short name in YAML, the class name must end with `GatewayFilterFactory`. If the class is named `JwtAuthFilter` instead of `JwtAuthGatewayFilterFactory`, the YAML reference `JwtAuth` will not resolve. I also verify that the filter class is in a package scanned by Spring and registered as a bean. If the route uses the filter with arguments, I check that the configuration class matches the YAML structure and that constructor binding is correct. Finally, I inspect the actuator gateway routes endpoint to see whether the filter is attached to the route and whether there are errors during route loading. Real-time example: a developer writes `AuditLogFilter` and registers it in YAML as `- AuditLog`. Locally, the YAML is small and the error may be silently ignored, but in production the

gateway fails to bind the route and the filter is skipped. Renaming the class to

`AuditLogGatewayFilterFactory` and using `- AuditLog` fixes the binding. I also check that the production build includes the package in component scanning; if the class is in a utility module, it may not be scanned by the gateway's main application.

S3. A global logging filter in the gateway logs every request, but it also logs sensitive tokens. How do you fix it?

Answer: I update the logging filter to sanitize or skip sensitive headers before logging. I maintain a deny list of headers such as `Authorization`, `Cookie`, `X-API-Key`, and `Refresh-Token`, and I replace their values with a mask such as `***` or remove them entirely. If the filter logs the request body, I apply the same redaction to fields known to contain passwords or tokens. I also consider logging only metadata such as method, path, status code, duration, trace ID, and client IP, and avoid logging payloads in production. For audit purposes that require full payloads, I send those to a separate secure system with access controls rather than to general application logs. Real-time example: during a security audit, the operations team discovers that raw `Authorization: Bearer eyJ...` tokens appear in CloudWatch Logs because the gateway logs the full request headers. This is a breach risk because anyone with log access can replay the token. I update the filter to iterate over headers, replace any header in the deny list with `[REDACTED]`, and add a review rule that any new logging filter must be checked for sensitive data exposure before merge.

S4. A Spring Cloud Config Server fails to start because it cannot clone the Git repository. How do you troubleshoot it?

Answer: I check the `spring.cloud.config.server.git.uri` property for typos or incorrect protocol. If the repository is private, I verify that credentials are supplied through properties, environment variables, or SSH keys and not hard-coded in the source. I also check network connectivity from the Config Server to GitHub, including proxy settings and firewall rules. If `clone-on-start` is true, the server tries to clone at startup, so any authentication or network issue is immediate. I set `clone-on-start` to false during troubleshooting to defer cloning, or I use a local file backend for development. I also look at the stack trace for HTTPS certificate errors, which can happen in environments with custom TLS interception. Real-time example: after a production deployment, the Config Server pod enters a `CrashLoopBackOff` because it cannot clone `https://github.com/acme/config-repo.git`. The repository was recently made private, but the deployment still uses the old public URI without credentials. I add a username and personal access token through Kubernetes secrets and reference them in `spring.cloud.config.server.git.username` and `password`, then restart the server. I also rotate the token and move to SSH or a GitHub App in the long term.

S5. A configuration value is updated in Git, but a microservice still uses the old value after calling `/actuator/refresh`. Why?

Answer: The bean that uses the property is probably not annotated with `@RefreshScope`. Only beans with `@RefreshScope` are recreated during a refresh event. If the value is read into a static field or initialized in a constructor without re-reading after refresh, the old value persists. Another cause is that the property file in Git was not matched by the client's application name or profile, so the Config Server never served the new value. I verify the Config Server endpoint for the exact `application/profile/label` to confirm the value is returned. I also check that the refresh actuator endpoint is exposed and that the call returned the changed property names. Real-time example: the team changes `feature.new-checkout.enabled` from `false` to `true` in `order-service-prod.yml` and calls `/actuator/refresh`. The feature flag service bean is not `@RefreshScope`, so it still reads the cached `false` value. I annotate the flag service with `@RefreshScope`, confirm the refresh response includes `feature.new-checkout.enabled`, and verify the new checkout UI appears for a small test segment before rolling out widely.

S6. A team wants to refresh configuration across 50 microservice instances without calling each one. How do you design it?

Answer: I use Spring Cloud Bus with RabbitMQ or Kafka as the message broker. When configuration changes, I configure a GitHub webhook to call the Config Server's `/monitor` endpoint, or I manually call `POST /actuator/busrefresh` on one instance. The bus publishes a refresh event that all connected clients consume, and each client refreshes its own `@RefreshScope` beans automatically. This avoids the operational burden of calling every instance. I also ensure the message broker is highly available because the refresh mechanism depends on it. For critical changes that require a restart, such as connection pool size changes at the JVM level, I use a rolling deployment instead of a refresh. Real-time example: during a sale event, marketing wants to update the maximum discount percentage across all services. Instead of SSH-ing into 50 pods, a developer pushes the new value to Git, the GitHub webhook hits `/monitor`, and Spring Cloud Bus broadcasts `RefreshRemoteApplicationEvent` to all `order-service`, `pricing-service`, and `cart-service` instances. Within seconds, every service picks up the new limit without a restart.

S7. A distributed trace shows spans from some services but is broken in the middle. How do you debug it?

Answer: I first identify which service is not propagating the trace context. I check whether the service uses an instrumented HTTP client; `RestTemplate` must be built with `RestTemplateBuilder`, `WebClient` with `WebClient.Builder`, and OpenFeign with Spring Cloud OpenFeign, otherwise trace headers are not added automatically. I also check whether the service has custom code that creates a new thread or uses a thread pool without copying the trace context. For async operations, I use `TaskDecorator` or reactor context propagation. I verify the propagation format; if one service uses W3C and another expects B3, the trace will be broken. Finally, I check that the service name is configured correctly so spans appear under the right service in Zipkin. Real-time example: a trace for `GET /orders/123` shows spans for the gateway, order service, and database, but the payment service span is missing. The payment service uses a manually created `RestTemplate` with `new RestTemplate()` instead of `RestTemplateBuilder`, so Micrometer Tracing does not inject the `traceparent` header. I switch to `RestTemplateBuilder`, verify the header is sent with a request test, and the next trace shows the full chain including the payment service and its downstream fraud check.

S8. A service logs are not appearing in Kibana even though Filebeat is running. What do you check?

Answer: I start by checking the Filebeat logs for connection errors, authentication failures, or parsing errors. I verify that Filebeat is configured to read the correct log path and that the application is actually writing logs to that path. I check whether Logstash or Elasticsearch is reachable from the Filebeat host and whether the index or data stream is configured. If the logs are in JSON but Filebeat expects plain text, or vice versa, parsing can fail silently. I also check Kibana index patterns to ensure they match the Elasticsearch indices where the logs are stored. In container environments, I verify that log rotation is not deleting files before Filebeat reads them. Real-time example: after deploying a new service, the team cannot find its logs in Kibana. Filebeat is configured to read `/var/log/app/*.log`, but the Spring Boot application logs to stdout and Docker sends output to `/var/lib/docker/containers/...`. I update the Filebeat input to use the Docker container log path or the `docker` input, add the `service.name` field, and the logs start appearing under the correct index pattern.

S9. A Spring Boot application writes logs in plain text, and the ELK stack cannot parse them consistently. How do you fix it?

Answer: I switch the application to structured JSON logging using a Logback or Log4j2 JSON encoder. JSON logs have consistent fields that Filebeat, Logstash, and Elasticsearch can parse without fragile regex patterns. I include fields such as timestamp, level, logger, message, service name, trace ID, span ID, and environment. I configure the logging pattern in `logback-spring.xml` or `log4j2-`

`spring.xml` and make it conditional on the active profile so local logs remain readable. After switching, I verify the mapping in Elasticsearch and update Kibana dashboards to use the new fields. Real-time example: one team writes logs like `2025-07-01 10:23:45 ERROR com.example.OrderService Failed to save order`. Another team uses `ERROR | OrderService | save failed`. Logstash has two grok filters, and both break when formats change. I standardize all services on JSON output with the `logstash-logback-encoder` so every log entry has the same `timestamp`, `level`, `logger`, `message`, `traceId`, and `service` fields, and dashboards become reliable.

S10. A Config Client crashes on startup because the Config Server is down. How do you make startup more resilient?

Answer: I make the config server import optional by using

`spring.config.import=optional:configserver:http://localhost:8888/`. With `optional`, the client starts using local configuration if the server is unavailable. I also configure retry with `spring.cloud.config.fail-fast=false` or a retry policy so the client attempts to connect multiple times before failing. For non-optional configurations, I run the Config Server in a high-availability setup with multiple instances behind a load balancer. I also add health checks and readiness probes in Kubernetes so pods are not marked ready until they can fetch configuration, preventing traffic from reaching misconfigured instances. Real-time example: during a regional outage, the Config Server in us-east-1 becomes unreachable, and new pods in the order service fail to start. I change the import to `optional:configserver` and include sensible defaults in `application.yml` so pods can start with degraded config. After the outage, I deploy a second Config Server in us-west-2 and use a DNS failover so clients always reach a healthy instance.

S11. A gateway filter must add different headers depending on the route. Should you use a `GlobalFilter` or a `GatewayFilterFactory`?

Answer: A `GatewayFilterFactory` is the better choice because it allows per-route configuration. The factory can accept a configuration object that specifies which headers to add, and the same filter can be used on different routes with different settings. A `GlobalFilter` applies to all routes and would need custom logic to inspect the route and decide what to do, which is harder to maintain. For route-specific transformations, custom `GatewayFilter` factories are cleaner, more declarative, and easier to test. I implement `AbstractGatewayFilterFactory` with a `Config` class and reference the filter by short name in each route's YAML. Real-time example: internal admin routes need `X-Internal: true`, while public API routes need `X-Public: true`. With a `HeaderInjectionGatewayFilterFactory`, I declare -

`HeaderInjection=headerName=X-Internal,headerValue=true` on admin routes and `HeaderInjection=headerName=X-Public,headerValue=true` on public routes. A global filter would need if-else logic based on path, making it harder to extend when a third route category is added.

S12. A microservice wants to change a log level in production without restarting. How do you do it?

Answer: I expose the Spring Boot Actuator `loggers` endpoint and send a `POST` request to change the level of a specific logger at runtime. For example, `POST /actuator/loggers/com.example.service` with a JSON body setting `configuredLevel` to `DEBUG` changes the level immediately. This does not require a restart and is useful for troubleshooting. I secure the endpoint so only authorized users can change log levels, and I document that these changes are temporary because they are lost on restart unless persisted. For a more centralized approach, I combine this with Spring Cloud Config and `/actuator/refresh` so log level changes are managed through configuration. Real-time example: a support ticket reports intermittent failures in the payment service. The on-call engineer temporarily sets `com.example.payment.PaymentClient` to `DEBUG` via the Actuator endpoint, captures a few failing requests, identifies a malformed downstream response, and then reverts the level to `INFO` once the issue is understood.

S13. A trace ID is present in the gateway logs but missing from database query logs inside a service. Why?

Answer: Database query logs are usually written by a separate library or connection pool that does not have access to the trace context stored in the request's MDC. If the query is executed synchronously during the request, the MDC should be populated, but some logging frameworks or JDBC drivers do not propagate MDC into their own log statements. I check whether the trace context is in the MDC when the database call is made and whether the logging pattern includes trace ID. For async database operations or reactive stacks, context propagation may be lost. I also consider using Micrometer Observation or a custom data source proxy that tags spans with database query information. Real-time example: the order service uses HikariCP and logs slow queries through a separate logger that does not include MDC. When a support engineer filters Kibana by trace ID `abc-123`, they see gateway and order service logs but no database entries. I wrap the data source with a custom proxy that copies MDC into the query thread, or I switch to Micrometer Observation so the database query becomes a span with the same trace ID.

S14. A Zipkin collector is receiving too many spans and becoming a bottleneck. How do you reduce load?

Answer: I reduce the sampling probability so only a percentage of requests are traced, for example `management.tracing.sampling.probability=0.1` in production. This gives enough visibility without overwhelming the collector. I also avoid creating spans for very high-frequency internal operations that are not useful for debugging. I batch and compress span reports, and I consider running multiple Zipkin collector instances behind a load balancer. For very large scale, I replace Zipkin with a more scalable backend such as Jaeger, Grafana Tempo, or a managed tracing service. Sampling is the first and most effective lever to reduce load. Real-time example: during a flash sale, the Zipkin server falls behind and drops spans because 100 percent sampling is enabled. I change the sampling probability to 0.05 for high-traffic endpoints and 1.0 for critical payment flows. This reduces volume by 95 percent while preserving visibility where it matters. I also add a queue between services and Zipkin to absorb bursts.

S15. A Config Server stores secrets in a Git repository in plain text. How do you secure them?

Answer: Storing secrets in plain text in Git is a serious risk. I use Spring Cloud Config's encryption and decryption features to store encrypted values, or I move secrets to a dedicated secret store such as HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault. Spring Cloud Config supports composite backends, so it can fetch configuration from Git and secrets from Vault. Another approach is to store secrets in environment variables provided by the deployment platform, while keeping non-secret configuration in Git. I also review Git history for leaked secrets and rotate any exposed credentials immediately. Real-time example: a security scan finds `spring.datasource.password=prod-password-123` in `order-service-prod.yml`. I rotate the password immediately, remove the value from Git history, and configure a composite backend that fetches the database password from Vault while keeping non-secret config in Git. Developers now see `${vault://secret/order-service/db/password}` instead of a plain password.

S16. A Spring Cloud Gateway route with a custom filter returns 500 for a specific request. How do you debug it?

Answer: I enable debug logging for `org.springframework.cloud.gateway` and inspect the stack trace. I check whether the filter is throwing an exception due to malformed input, missing configuration, or a downstream failure. Because the gateway is reactive, I look for blocking calls on the event loop, which manifest as warnings or deadlocks under load. I use the actuator gateway routes endpoint to verify the route and filter configuration, and I add unit tests for the filter in isolation. If the

issue is intermittent, I correlate gateway logs with downstream service logs using the trace ID to see whether the failure originates in the backend. Real-time example: a custom rate-limiting filter throws `NullPointerException` for requests that do not have a specific query parameter because the key resolver assumes the parameter is always present. I add a default value, write a unit test with and without the parameter, and update the filter to return 400 instead of 500 for invalid input.

S17. A centralized logging dashboard shows logs from some services with different timestamp formats. How do you standardize them?

Answer: I standardize the log output across all services by using a common logging configuration that writes timestamps in ISO-8601 UTC format, such as `yyyy-MM-dd'T'HH:mm:ss.SSSZ`. I include the timezone explicitly or use UTC to avoid ambiguity. If services use different logging frameworks, I configure each to match the same pattern or, better, switch all services to JSON logging with a common schema. In Logstash, I use date filters to parse incoming timestamps and store them in a consistent `@timestamp` field. Standardized timestamps are critical for sorting and correlating events across services in Kibana. Real-time example: one legacy service logs in local IST, a new service logs in UTC, and the gateway uses a custom format. When debugging an incident, the team cannot sort events chronologically. I migrate all services to UTC ISO-8601 in JSON and add a Logstash date filter that normalizes every incoming timestamp to `@timestamp` before indexing.

S18. A team wants to use Spring Cloud Config but also needs local overrides per developer machine. How do you support both?

Answer: I use the configuration loading order to my advantage. Spring Cloud Config provides shared and application-specific configuration, and local `application.yml` or environment variables can override remote values. I give developers a default remote configuration in Git and allow them to create a local profile or use environment variables for personal overrides. I keep the `spring.config.import` for the Config Server but make it optional so developers can run offline. I also use `.gitignore` for local override files and document the override precedence so developers understand that local properties win over remote ones. Real-time example: developers want to use a local MySQL container while the shared config points to a cloud RDS instance. I add `spring.datasource.url` to `application-local.yml` in each developer's project, ignored by Git, and tell them to run with `--spring.profiles.active=local`. The shared `application.yml` still points to the cloud database for CI and production.

S19. A gateway filter needs to call an external service for enrichment before forwarding the request. What do you consider?

Answer: Calling an external service from a gateway filter adds latency and a failure point to every request. I consider whether the call can be avoided by caching the enrichment data or by pushing the responsibility to the downstream service. If the call is necessary, I make it reactive and non-blocking, and I add a timeout and a circuit breaker around it so a slow external service does not degrade the gateway. I also use a bulkhead to limit how many concurrent enrichment calls can run. The filter should handle failures gracefully, either by forwarding without enrichment or by returning an error, depending on business requirements. Real-time example: a gateway filter calls a user-profile service to add a loyalty tier header to every request. When the profile service slows down, gateway latency increases and threads are exhausted. I add a 200-millisecond timeout and a circuit breaker around the call, cache the loyalty tier in Redis with a 5-minute TTL, and return a default tier when the service is unavailable instead of failing the entire request.

S20. A distributed system has services in multiple programming languages. How do you ensure trace propagation works across all of them?

Answer: I standardize on a propagation format supported by all services, typically W3C trace context because it is language-agnostic and widely supported. Each service extracts the `traceparent` header from incoming requests and injects it into outgoing requests. For services that do not have native tracing libraries, I instrument them with OpenTelemetry, which supports many languages. I also use a service mesh such as Istio or Linkerd, which can inject and propagate trace headers transparently without changing application code. The gateway or ingress layer can be configured to accept incoming trace headers and ensure they are forwarded to the first service in the system. Real-time example: a Python-based recommendation service sits between Java order and notification services. The Java services use Micrometer Tracing with B3, but the Python service uses OpenTelemetry with W3C. I configure the Java services to propagate both B3 and W3C headers and update the Python service to read whichever is present. Over time, I migrate everything to W3C for consistency.